pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Case Study: The Market Dataset

WEEK 1 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem definition: one week ends, one dataset begins

The Desk Lead's Fifth Note

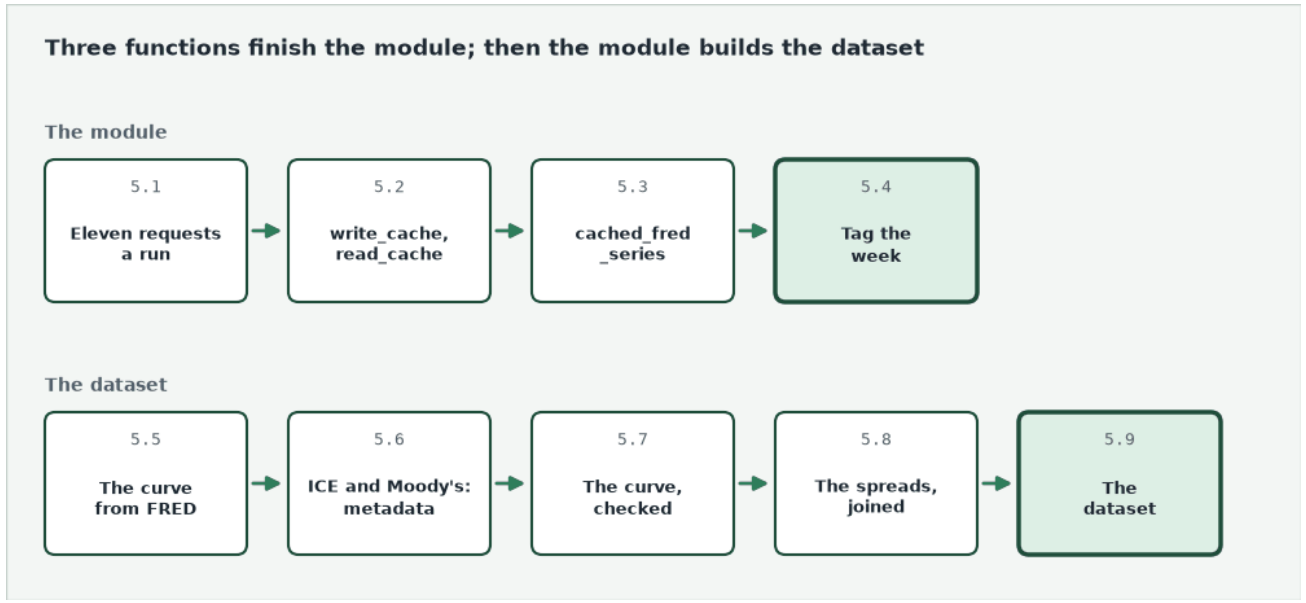
Week 1 ends today, and weeks 2 and 3 start from one dataset. First the pulls: a morning of reruns must not send FRED hundreds of requests, so keep a copy of each series for the day, and prove the copy gives back every digit. ICE and Moody's series must never be stored: make the code refuse them, not a habit. Show me the same curve comes from FRED through that copy, equal to our file, and tag the week. Then build the daily dataset, five Treasury tenors and two credit spreads, one row per market day, and prove it clean with every check of the week. Describe the data: no view on any rate or spread.

You are the desk's analyst: answer the five asks with your own "marketdata.py", and make every check an "assert".

The Data

File	What it holds	Units	Licence
`treasury_par_yields.csv`	Treasury par yields, 14 tenors, 2,940 market days	percent	Public domain
`course3_synthetic_spreads.csv`	ig_spread_bp and hy_spread_bp on the same dates	bp	Synthetic: invented for this course, not ICE, Moody's, or any index

How we will work, and the module this morning



Three functions finish week 1's module; then the module builds the dataset.

The Module as Notebook 04 Left It

```
.................   [100%]
17 passed in 0.50s
```

Today's Work Folder

```
work = start_day(5)
Work folder pfi_course3_05_k4w7q2xr: 4 data files, Course 2's bondmath.py, marketdata.py as notebook 04 left it, 4 commits
```

5.1 Eleven requests a run

Two Constants: the Curve's Series, and the IDs Never Stored

```
%%writefile -a marketdata.py
from datetime import date, datetime  # the cache file's date, from day 5
FRED_TREASURY = {
    "DGS1MO": "1 Mo", "DGS3MO": "3 Mo", "DGS6MO": "6 Mo", "DGS1": "1 Yr", "DGS2": "2 Yr", "DGS3": "3 Yr",
    "DGS5": "5 Yr", "DGS7": "7 Yr", "DGS10": "10 Yr", "DGS20": "20 Yr", "DGS30": "30 Yr",
}
LICENSED_PREFIXES = ("BAML", "BAA", "AAA", "DBAA", "DAAA")
```

What a Morning Costs

```
importlib.reload(md)
requests_per_run = len(md.FRED_TREASURY)
print(f'Twelve runs: {12 * requests_per_run} requests without a cache, '
      f'{requests_per_run} with one')
Twelve runs: 132 requests without a cache, 11 with one
```

Notebook 02's requests chart showed where quick uncached runs lead: the eleventh passes FRED's 120 requests a minute. A copy on disk is also faster, and still there when FRED is down.

Without a cache every rerun costs 11 more requests; with one, the whole day costs 11.

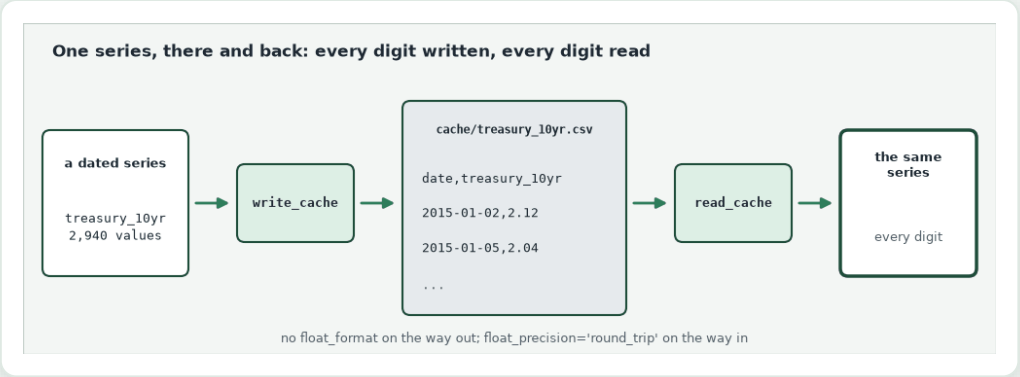
5.2 A copy that gives back every digit

write_cache and read_cache, Appended

```
%%writefile -a marketdata.py
def write_cache(series: pd.Series, cache_dir: Path) -> Path:
    _check_index(series.index, "series")
    if series.name is None:
        raise ValueError("series must have a name: it becomes the file name")
    cache_dir = Path(cache_dir)
    cache_dir.mkdir(parents=True, exist_ok=True)
    path = cache_dir / f"{series.name}.csv"
    series.to_csv(path, index_label="date", lineterminator="\n")
    return path
def read_cache(series_id: str, cache_dir: Path) -> pd.Series:
    path = Path(cache_dir) / f"{series_id}.csv"
    if not path.is_file():
        raise FileNotFoundError(f"no cache file {path.name} in the folder {Path(cache_dir).name}")
    frame = pd.read_csv(path, parse_dates=["date"], index_col="date", float_precision="round_trip")
    return frame[series_id].astype("float64")
```

Cache the 10 Yr

```
importlib.reload(md)
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
ten_year = treasury['10 Yr'].rename('treasury_10yr')
cache_dir = work / 'cache'
cached_path = md.write_cache(ten_year, cache_dir)
print(cached_path.read_text(encoding='utf-8')[:50])
date,treasury_10yr
2015-01-02,2.12
2015-01-05,2.04
```



The cache is a folder, "cache/", one CSV per series. Notebook 01's ".gitignore" already keeps "cache/" out of Git.

"mkdir(parents=True, exist_ok=True)" makes the folder and any folder above it, and is quiet if it exists. "index_label=" names the date column, and "lineterminator=" sets the line ending.

Read It Back

```
back = md.read_cache('treasury_10yr', cache_dir)
pd.testing.assert_series_equal(back, ten_year, check_exact=True)
print(f'{len(back):,} of {len(ten_year):,} values back, every one exact')
2,940 of 2,940 values back, every one exact
```

A file that is not there, and two tests

Ask for a File That Is Not There

```
md.read_cache('DGS2', cache_dir)
FileNotFoundError: no cache file DGS2.csv in the folder cache
```

An error, never an empty series: an empty series would look like a series with no data.

Two Tests

```
%%writefile -a test_marketdata.py
def test_cache_round_trip_exact(tmp_path):
    series = md.parse_fred_observations(read_sample("DGS10"), "DGS10")
    path = md.write_cache(series, tmp_path / "cache")
    assert path.name == "DGS10.csv"
    back = md.read_cache("DGS10", tmp_path / "cache")
    pd.testing.assert_series_equal(back, series, check_exact=True)
    awkward = (series.dropna() / 3).rename("THIRDS")
    md.write_cache(awkward, tmp_path / "cache")
    pd.testing.assert_series_equal(md.read_cache("THIRDS", tmp_path / "cache"), awkward, check_exact=True)
def test_read_cache_missing_raises(tmp_path):
    with pytest.raises(FileNotFoundError):
        md.read_cache("DGS10", tmp_path)
```

Run pytest

```
..................   [100%]
19 passed in 0.50s
```

Every digit comes back because "read_cache" asks for every digit. Next, the function that decides whether to ask FRED at all.

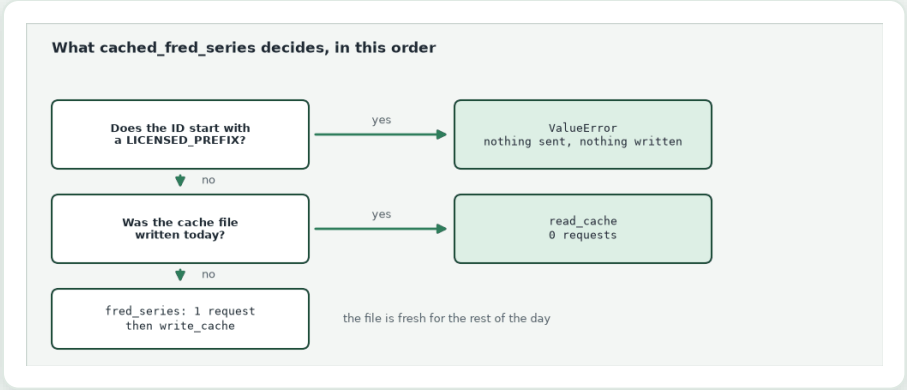
Excel side by side: the 2,940 thirds, opened

Job	Python	Excel	Result
Open a CSV of the 2,940 thirds	`md.read_cache(...)`	File, Open	Python 0 changed; Excel 2,226, the largest by 9.8e-15

5.3 cached_fred_series refuses licensed series

Append It, Docstring First

```
%%writefile -a marketdata.py
def cached_fred_series(series_id: str, api_key: str, cache_dir: Path, start: str | None = None) -> pd.Series:
    """A FRED series from today's cache file if there is one, otherwise from FRED, then cached.
    Units: as FRED publishes the series. Convention: the cache file counts as fresh if it was written today
    (the computer's local date); otherwise fred_series downloads the series from `start` and write_cache
    saves it. A fresh file is returned as it is, whatever `start` it was downloaded with.
    Raises ValueError for a series ID that starts with one of LICENSED_PREFIXES: those providers' terms do
    not allow storing their data. The list covers this course's licensed series; it is not a general
    licence check, and any other series' terms are yours to read.
    """
    if series_id.startswith(LICENSED_PREFIXES):
        raise ValueError(f"{series_id} is licensed data (prefix in LICENSED_PREFIXES): its terms do not allow "
                         f"storing it, so it is never cached. Use fred_series and keep it in memory.")
    path = Path(cache_dir) / f"{series_id}.csv"
    if path.is_file() and datetime.fromtimestamp(path.stat().st_mtime).date() == date.today():
        return read_cache(series_id, cache_dir)
    series = fred_series(series_id, api_key, start=start)
    write_cache(series, cache_dir)
    return series
```



It may send a request, so it runs only in 5.5's live cells; its two rules show without the network.

The refusal comes first, before a request or a file.

Fresh, refused, and a test that sends nothing

Fresh Means Written Today

```
from datetime import date, datetime
written = datetime.fromtimestamp(cached_path.stat().st_mtime).date()
print(f'Written today: {written == date.today()}')
yesterday = cached_path.stat().st_mtime - 24 * 60 * 60
os.utime(cached_path, (yesterday, yesterday))
written = datetime.fromtimestamp(cached_path.stat().st_mtime).date()
print(f'After moving it back a day: {written == date.today()}')
Written today: True
After moving it back a day: False
```

"datetime.fromtimestamp" turns seconds into a date and time, and "date.today()" is the computer's date. Tomorrow the whole cache is old, and each series is pulled once.

The Refused IDs

```
importlib.reload(md)
for series_id in ['DGS10', 'DGS2', 'BAMLC0A0CM', 'BAMLH0A0HYM2', 'BAA10Y', 'DBAA']:
    refused = series_id.startswith(md.LICENSED_PREFIXES)
    print(f"{series_id:13} {'refused: licensed' if refused else 'may be cached'}")
DGS10         may be cached
DGS2          may be cached
BAMLC0A0CM    refused: licensed
BAMLH0A0HYM2  refused: licensed
BAA10Y        refused: licensed
DBAA          refused: licensed
```

The Test That Proves No Request Was Sent

```
%%writefile -a test_marketdata.py
def test_licensed_series_not_cached(monkeypatch, tmp_path):
    def no_network(*args, **kwargs):
        raise AssertionError("cached_fred_series must not send a request for a licensed series")
    monkeypatch.setattr(requests, "get", no_network)
    for series_id in ("BAMLC0A0CM", "BAMLH0A0HYM2", "BAA10Y"):
        with pytest.raises(ValueError, match="licensed"):
            md.cached_fred_series(series_id, FAKE_KEY, tmp_path / "cache")
    assert not (tmp_path / "cache").exists()
```

"monkeypatch.setattr" swaps in the stand-in, as in notebook 02. "match=" also checks the error's message holds the word.

Run pytest

```
....................   [100%]
20 passed in 0.50s
```

No request, no file: a check that stops beats a habit. Week 1's module is complete.

5.4 Tag the week

What Git Sees

```
!git -C "{work}" status --short --ignored
!git -C "{work}" check-ignore -v cache/treasury_10yr.csv
 M marketdata.py
 M test_marketdata.py
!! __pycache__/
!! bondmath.py
!! cache/
!! course3_excel_reference.csv
!! course3_fred_format_dgs10.json
!! course3_synthetic_spreads.csv
!! treasury_par_yields.csv
.gitignore:4:cache/    cache/treasury_10yr.csv
```

"cache/" is ignored by line 4 of notebook 01's ".gitignore". The data files and "bondmath.py" are copies the setup put on Git's local ignore list, so they show "!!" too.

Read the Diff, Commit

```
!git -C "{work}" diff --stat
subject = 'Notebook 05: write_cache, read_cache, and cached_fred_series'
body = 'Week 1 ends: one request per series per day, and licensed data stays in memory.'
!git -C "{work}" add marketdata.py test_marketdata.py
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
 marketdata.py   | 70 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 31 +++++++++++++++++++++++++
 2 files changed, 101 insertions(+)
```

Name the Week

```
!git -C "{work}" tag week1
!git -C "{work}" log --oneline --decorate
ba73980 (HEAD -> main, tag: week1) Notebook 05: write_cache, read_cache, and cached_fred_series
9f01b1c Notebook 04: last_per_period and value_as_of
a439502 Notebook 03: missing_weekdays, align, and stale_runs
ceea769 Notebook 02: fred_params, parse_fred_observations, and fred_series
a28686d Notebook 01: get_secret, a .gitignore, and .env.example
```

"main" moves on with tomorrow's commit; "week1" stays here, as Course 2's picture of a tag showed.

"week1" names the module as the week ends: three functions, three tests, and a cache Git never sees. Now the copy on real data.

5.5 Live: the curve from FRED, through the cache

This product uses the FRED® API but is not endorsed or certified by the Federal Reserve Bank of St. Louis.

The Key Cell

```
from pathlib import Path
ENV_FILE = Path.home() / "projects" / "my_bondmath" / ".env"
NO_KEY_MESSAGE = "Live cell skipped: no FRED key was found. Every other cell runs without one."
SHOW_LICENSED_VALUES = False
try:
    fred_key = md.get_secret("FRED_API_KEY", env_file=ENV_FILE)
    print("FRED key: found. It is held in memory and is never printed.")
except RuntimeError as error:
    fred_key = None
    print(f"FRED key: not found. Live cells will be skipped. {error}")
FRED key: not found.
Live cells will be skipped.
Secret FRED_API_KEY not found.
Looked in: the environment variable FRED_API_KEY, Colab's Secrets panel, and the file .env given as env_file.
```

Eleven Series, Checked Against the File

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    live_cache = work / 'cache'
    live_start = '2015-01-01'
    for live_id, live_tenor in md.FRED_TREASURY.items():
        live_series = md.cached_fred_series(live_id, fred_key, live_cache, start=live_start)
        live_series = live_series.dropna()
        live_common = live_series.index.intersection(treasury.index)
        live_gap = (live_series[live_common] - treasury.loc[live_common, live_tenor]).abs().max()
        assert len(live_common) > 0 and live_gap <= 1e-9, f'{live_id} differs from the file'
        print(f'{live_id:7} {live_tenor:6} {len(live_common):,} dates in both, '
              f'largest gap {live_gap:.1e}')
    for live_id in md.FRED_TREASURY:
        live_again = md.cached_fred_series(live_id, fred_key, live_cache, start=live_start)
        assert live_again.equals(md.read_cache(live_id, live_cache))
    print(f'Second pass: {len(md.FRED_TREASURY)} series from {live_cache.name}/, no request sent')
Live cell skipped: no FRED key was found. Every other cell runs without one.
```

With your key: Eleven lines, each with a largest gap of "0.0e+00" percent: FRED's "DGS" series hold the Treasury file's published values. The second pass reads eleven files and sends nothing.

The cache works on the real thing. FRED carries credit series too.

5.6 ICE and Moody's on FRED: metadata only

Three series a credit desk knows are on FRED: ICE BofA's investment grade and high yield option-adjusted spreads, "BAMLC0A0CM" and "BAMLH0A0HYM2", and Moody's Baa yield less the 10-year Treasury, "BAA10Y". A free key downloads them; the providers own them.

Series	What the notes say (read 2026-10-04)
ICE BofA	"Starting in April 2026, this series will only include 3 years of observations."
ICE BofA	"... being provided for your internal use only
	and you are not authorized or permitted to publish, distribute or otherwise furnish Top Level Data to any third-party ..."
ICE BofA	"When the last calendar day of the month takes place on the weekend,
	weekend observations will occur as a result of month ending accrued interest adjustments."
Moody's	"MOODY'S INFORMATION MAY NOT BE COPIED OR OTHERWISE REPRODUCED ... OR STORED FOR SUBSEQUENT USE ... WITHOUT MOODY'S PRIOR WRITTEN CONSENT."

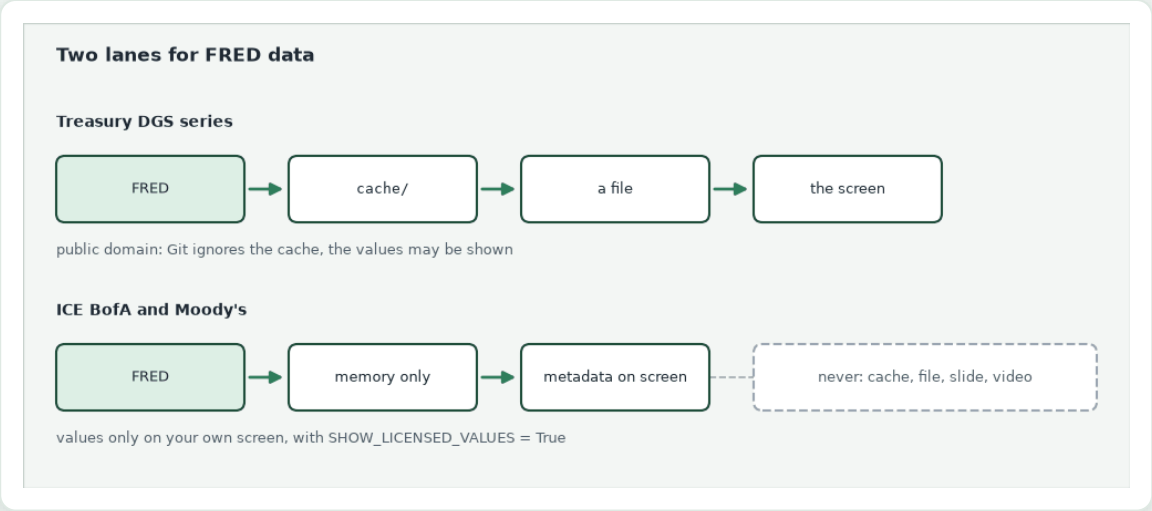
Sources: https://fred.stlouisfed.org/series/BAMLC0A0CM and https://fred.stlouisfed.org/series/BAA10Y. FRED's terms add that a series whose notes say "Copyright" is owned by a third party (https://fred.stlouisfed.org/docs/api/terms_of_use.html).

Three Years, and No More

Three years, and no more. In April 2026 ICE BofA's series on FRED were cut to three years of history. When this course was written, a request to one of them for 2010 came back with HTTP status 200 and zero observations: an empty answer, not an error. "fred_series" raises on exactly that, as notebook 02 tested.

Source: FRED series notes, https://fred.stlouisfed.org/series/BAMLC0A0CM (read 2026-10-04).

Pulled into memory, described, never cached



A saved notebook, a slide and a video all publish, so the live cell prints metadata only: rows, first and last date, "." rows, weekend rows.

Licensed values never reach a cache, a file, a slide, or a video, so the dataset's credit columns are synthetic, and labelled so. Now the case study.

Pulled Into Memory, Described

```
if fred_key is None:
    print(NO_KEY_MESSAGE)
else:
    for live_id in ['BAMLC0A0CM', 'BAMLH0A0HYM2', 'BAA10Y']:
        live_series = md.fred_series(live_id, fred_key)
        live_weekend = (live_series.index.dayofweek >= 5).sum()
        print(f'{live_id}: percent, {len(live_series):,} rows, {live_series.index[0].date()} to '
              f'{live_series.index[-1].date()}, {live_series.isna().sum()} . rows, '
              f'{live_weekend} weekend rows')
        assert live_series.dropna().between(0, 30).all(), f'{live_id}: a value outside 0 to 30'
        if live_id.startswith('BAML'):
            live_floor = pd.Timestamp.today() - pd.DateOffset(years=3, months=1)
            assert live_series.index[0] >= live_floor, f'{live_id}: more than three years'
        if SHOW_LICENSED_VALUES:
            print((live_series.dropna() * 100).tail(5).round(1))
            (live_series * 100).plot(title=f'{live_id} in bp (licensed: your screen only)')
            plt.show()
    for live_path in (work / 'cache').glob('*.csv'):
        assert not live_path.name.startswith(md.LICENSED_PREFIXES), f'{live_path.name} is licensed'
    print('Cache: no licensed series written')
Live cell skipped: no FRED key was found. Every other cell runs without one.
```

With your key: One line per series: about three years of daily ICE rows, with weekend rows at month ends, and a longer "BAA10Y"; then the cache line. Values appear only with "SHOW_LICENSED_VALUES = True" on your own machine: keep them to yourself.

5.7 The curve, checked

Dates: Unique, Ascending, Weekdays

```
assert treasury.index.is_unique and treasury.index.is_monotonic_increasing, 'dates out of order'
weekend_rows = (treasury.index.dayofweek >= 5).sum()
assert weekend_rows == 0, 'a weekend date'
print(f'{len(treasury):,} dates, unique and ascending, {weekend_rows} on a weekend')
2,940 dates, unique and ascending, 0 on a weekend
```

The Market's Own Calendar

```
missing = md.missing_weekdays(treasury.index)
assert len(missing) == 126, 'the missing weekdays changed since notebook 03'
print(f'{len(missing)} weekdays with no row, the last {missing[-1].date()}')
126 weekdays with no row, the last 2026-09-07
```

Job	Python	Excel	Result
Weekend dates	`(treasury.index.dayofweek >= 5).sum()`	`=SUMPRODUCT(--(WEEKDAY(A2:A2941,2)>5))`	0
Repeated dates	`treasury.index.is_unique`	`=ROWS(A2:A2941)-ROWS(UNIQUE(A2:A2941))`	0
Dates out of order	`treasury.index.is_monotonic_increasing`	`=SUMPRODUCT(--(A3:A2941<=A2:A2940))`	0

Three Tenors Start Late

```
first_dates = pd.Series({tenor: treasury[tenor].first_valid_index() for tenor in treasury.columns})
for tenor, start in first_dates[first_dates > treasury.index[0]].items():
    blanks = treasury.loc[:start, tenor].isna().sum()
    gaps = treasury.loc[start:, tenor].isna().sum()
    print(f'{tenor:9} from {start.date()}: {blanks:,} blank rows before, {gaps} after')
```

Excel side by side: the 4 Mo

Job	Excel	Result
Blank cells of the 4 Mo, column F	`=COUNTBLANK(F2:F2941)`	1,951
The 4 Mo's first date	`=XLOOKUP(TRUE,F2:F2941<>"",A2:A2941)`	2022-10-19

A blank before a tenor starts is not a missing value: nothing was published. The dataset takes five tenors that run the whole file.

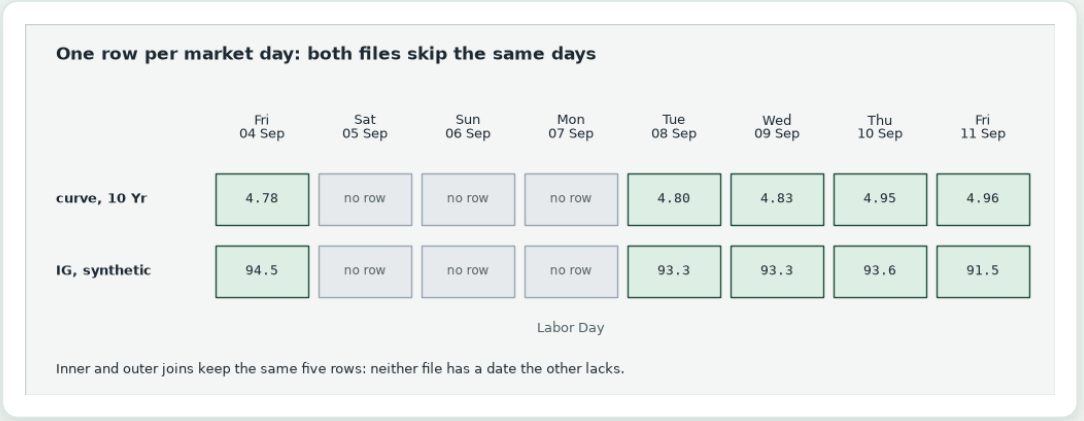
5.8 The spreads, joined

The Second File

```
spreads = pd.read_csv('course3_synthetic_spreads.csv', parse_dates=['date'], index_col='date',
                      float_precision='round_trip')
pair = {'10 Yr': treasury['10 Yr'], 'ig_spread_bp': spreads['ig_spread_bp']}
inner, outer = md.align(pair, how='inner'), md.align(pair, how='outer')
print(f'inner {len(inner):,} rows, outer {len(outer):,} rows, {outer.isna().sum().sum()} blank')
inner 2,940 rows, outer 2,940 rows, 0 blank
```

Two Files, Stacked and Joined

```
stacked = pd.concat([treasury.loc[:'2025'], treasury.loc['2026':]])
merged = treasury.join(spreads)
print(f'stacked {len(stacked):,} rows, equal to the file: {stacked.equals(treasury)}')
print(f'merged {len(merged):,} rows, {merged.shape[1] + 1} columns with the date')
stacked 2,940 rows, equal to the file: True
merged 2,940 rows, 17 columns with the date
```



Excel side by side. Power Query, Data, Get Data, Combine Queries:

Job	Python	Excel	Result
Stack 2,750 and 190 rows	`pd.concat`	Append: Table.Combine({First, Second})	2,940 rows, 15 columns
Join on the date	`treasury.join(spreads)`	Merge: Table.NestedJoin(... JoinKind.LeftOuter), then	2,940 rows, 17 columns, spreads as text (98.8, 334.7) until
		Table.ExpandTableColumn	Table.TransformColumnTypes(..., "en-US")
A spread beside each date	`md.align`	=XLOOKUP($A2,Spreads!$A$2:$A$2941,Spreads!$B$2:$B$2941), filled down	0 #N/A

5.9 The dataset, built and checked

Seven Columns, One Row per Market Day

```
columns = {tenor: treasury[tenor] for tenor in TENORS}
columns['ig_spread_bp'] = spreads['ig_spread_bp']
columns['hy_spread_bp'] = spreads['hy_spread_bp']
dataset = md.align(columns, how='inner')
print(f'{len(dataset):,} rows, {dataset.shape[1]} columns')
2,940 rows, 7 columns
```

Every Check of Week 1

```
assert dataset.index.is_unique and dataset.index.is_monotonic_increasing, 'dates out of order'
assert (dataset.index.dayofweek >= 5).sum() == 0, 'a weekend date'
assert md.missing_weekdays(dataset.index).equals(missing), 'the market calendar changed'
assert dataset.notna().all().all(), 'a blank cell'
print('Unique ascending dates, no weekend, the same 126 missing weekdays, no blank')
Unique ascending dates, no weekend, the same 126 missing weekdays, no blank
```

Job	Python	Excel	Result
Blank looked-up spreads, column P	`dataset.notna().all()`	`=COUNTBLANK(P2:P2941)`	0

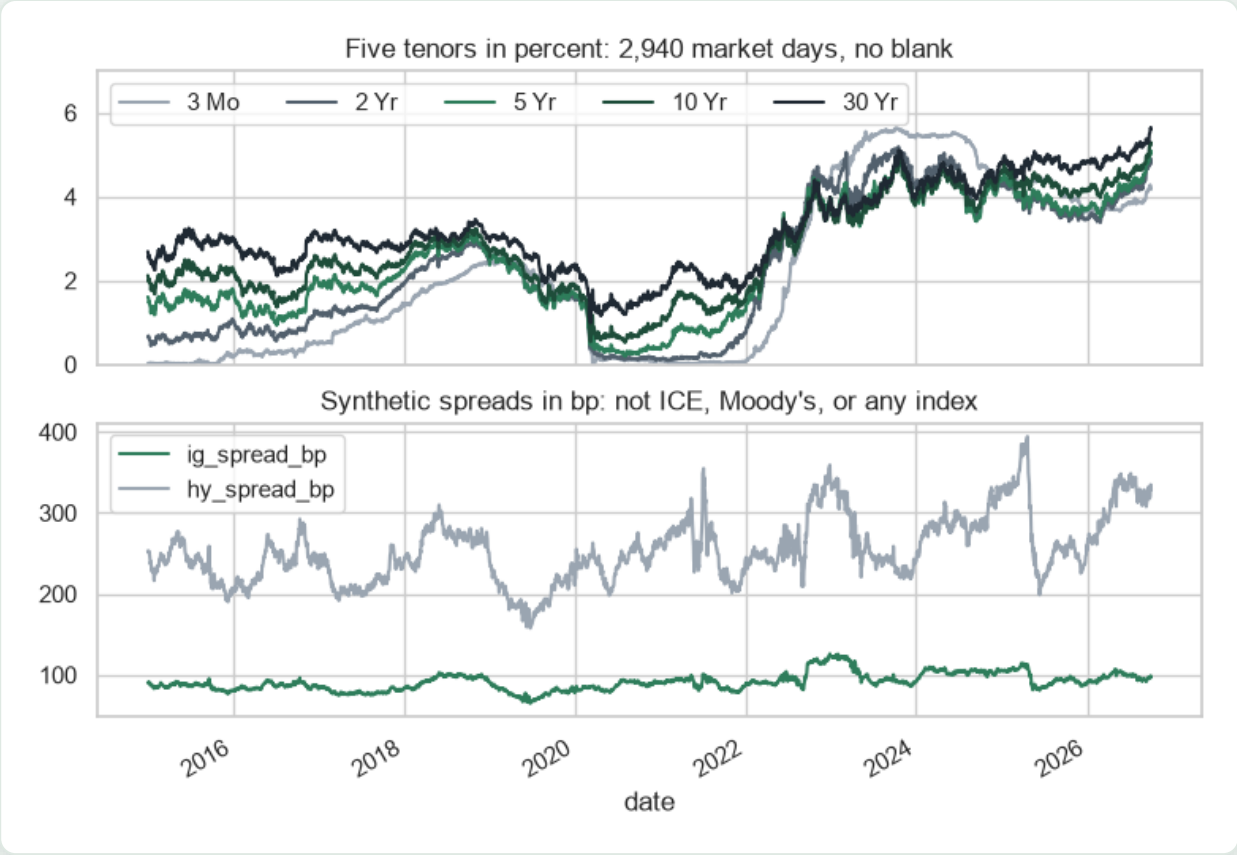
On 2026-09-30 the 10 Yr is 5.29 percent, notebook 04's month-end value. The synthetic spreads were set to end there at 98.8 and 334.7 bp, the market-value weighted OAS of the investment grade and high yield bonds in Course 2's September book. October's row is the file's last day so far, 2026-10-02.

Month Ends of 2026

```
month_ends = md.last_per_period(dataset.loc['2026'], 'ME')
(month_ends.style.format('{:.2f}', subset=TENORS).format('{:.1f}', subset=SPREADS)
    .format_index('{:%Y-%m-%d}')
    .background_gradient(cmap='Greens', subset=['10 Yr', 'ig_spread_bp', 'hy_spread_bp']))
```

date	3 Mo	2 Yr	5 Yr	10 Yr	30 Yr	ig_spread_bp	hy_spread_bp
2026-01-30	3.67	3.52	3.79	4.26	4.87	91.3	267.2
2026-02-27	3.67	3.38	3.51	3.97	4.64	95.6	287.5
2026-03-31	3.70	3.79	3.92	4.30	4.88	96.5	288.1
2026-04-30	3.68	3.88	4.02	4.40	4.98	106.8	338.1
2026-05-29	3.69	3.98	4.13	4.45	4.99	102.2	334.9
2026-06-30	3.87	4.14	4.19	4.44	4.91	100.1	345.4
2026-07-31	3.83	4.28	4.45	4.75	5.27	97.2	327.1
2026-08-31	3.91	4.34	4.49	4.75	5.25	94.3	319.9
2026-09-30	4.20	4.88	5.09	5.29	5.64	98.8	334.7
2026-10-02	4.19	4.83	5.06	5.28	5.63	97.0	328.7

The dataset, drawn, and the summary



Summary

Check	Result
The 10 Yr through the cache	2,940 of 2,940 exact
Licensed IDs	refused, no request, no folder
pytest	20 passed
Dates	2,940 unique, ascending, 0 on a weekend
Missing weekdays	126, as notebook 03
Late tenors	3, none in the dataset
Spreads joined	inner 2,940, outer 2,940
Dataset	2,940 rows, 7 columns, 0 blank

Bring it home: into my_bondmath, tagged

Copy the two files into your own folder, test, commit, and tag the week there. In a PowerShell terminal in VS Code with the course's ".venv" active, use the work folder name your setup cell printed:

```
$work = "$env:TEMP\pfi_course3_05_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git check-ignore -v cache/DGS10.csv
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 05: write_cache, read_cache, and cached_fred_series"
git tag week1
git log --oneline --decorate -3
```

The copy replaces your two files, so anything of your own goes into the work folder first. The data files the tests read are already there. "check-ignore" must name notebook 01's "cache/" line; if it prints nothing, add the line before you commit. With your key, 5.5's loop with "live_cache" set to the "cache" folder in "my_bondmath" pulls each series once a day. In Colab, download the two files from the work folder under "/tmp". The work folder stays in your temp folder until you delete it.

You Can Now

Say what a cache saves: FRED's limit, time, and a pull when FRED is down.

Cache a series and read it back exactly, with float_precision="round_trip".

Use "cached_fred_series", which pulls at most once a day and refuses a licensed series before sending anything.

Keep ICE's and Moody's values off every saved or shared screen.

Build a daily market dataset and prove it clean with every check of week 1.

Tag the end of a week in Git and find the tag with "git log --decorate".

Predict first: the default reader, twice

Predict 1

The 10 Yr has two decimals. Cache it divided by 3 instead, with every digit a float holds, and read it back with plain "pd.read_csv", no "float_precision". How many of the 2,940 values come back different? Write down a number.

```
thirds = (treasury['10 Yr'] / 3).rename('treasury_10yr_thirds')
md.write_cache(thirds, cache_dir)
plain = pd.read_csv(cache_dir / 'treasury_10yr_thirds.csv', parse_dates=['date'], index_col='date')
default_wrong = (plain['treasury_10yr_thirds'] != thirds).sum()
exact_wrong = (md.read_cache('treasury_10yr_thirds', cache_dir) != thirds).sum()
print(f'pandas default: {default_wrong} differ; read_cache: {exact_wrong} differ')
```

Predict 2

The 10 Yr as first cached, two decimals, through the same default reader. How many differ?

A) None

B) About the same share as the thirds

C) A few dozen

```
plain_two = pd.read_csv(cache_dir / 'treasury_10yr.csv', parse_dates=['date'], index_col='date')
print(f"two decimals: {(plain_two['treasury_10yr'] != ten_year).sum()} of 2,940 differ")
```

Predict first: full rows, and the stillest column

Predict 3

The file has 14 tenors and 2,940 rows. On how many rows does every tenor have a value? Write down a number.

```
print(f'{len(treasury.dropna()):,} rows have all 14 tenors')
```

Predict 4

Yields are published to 0.01 percent, the synthetic spreads to 0.1 bp. Which of the dataset's seven columns has the most runs of 5 or more equal values?

A) A synthetic spread

B) The 3 Mo

C) The 30 Yr

```
TENORS = ['3 Mo', '2 Yr', '5 Yr', '10 Yr', '30 Yr']
SPREADS = ['ig_spread_bp', 'hy_spread_bp']
runs_of_five = pd.Series({column: len(md.stale_runs(merged[column], min_length=5))
                          for column in TENORS + SPREADS})
assert runs_of_five[SPREADS].sum() == 0, 'a synthetic spread went stale'
print(f'The {runs_of_five.idxmax()} has the most: {runs_of_five.max()}')
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: build_market_dataset's test, written from its docstring, fails: AssertionError: 2026-09-30 does not hold that day's 10 Yr.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
A day's copy of each series	cached_fred_series: 11 requests a morning, every digit back
Never store ICE or Moody's	Refused in code before any request, and tested
The curve from FRED	Equal to the file on every common date, with your key
The week tagged	week1
The daily dataset	2,940 market days, five tenors, two synthetic spreads, every check passed

Be proud: a desk trusts a number because someone checked what sits under it, and this week you built the checks.

Next, Changes in bp and Percent (notebook 06) measures what the data did, starting with a change in basis points.